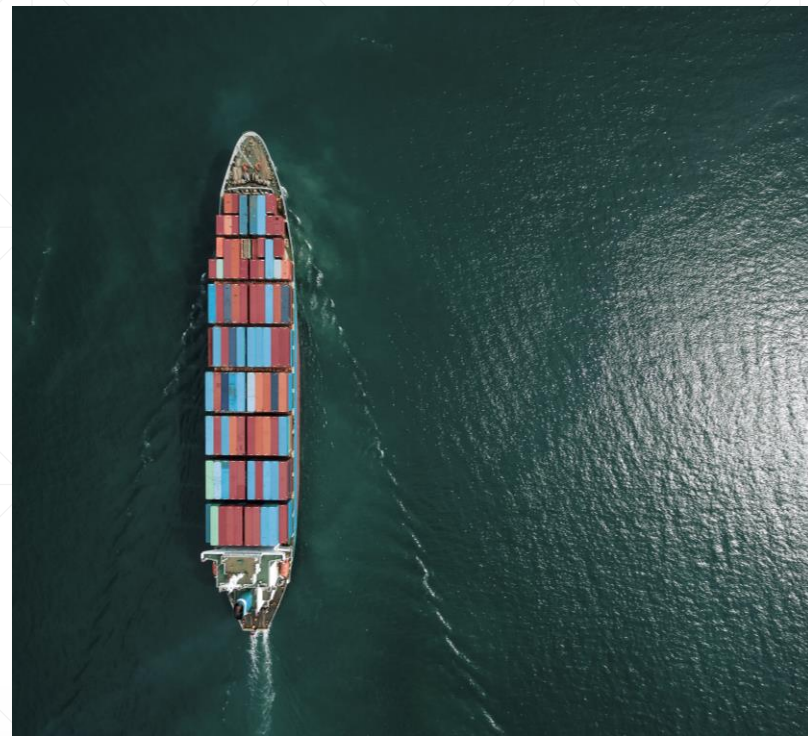




Stream API概述

北京理工大学计算机学院
金旭亮

引例

假设我们有一个学生对象的集合，希望知道所有来自于北京的学生人数。
传统的编程方式是这样的：

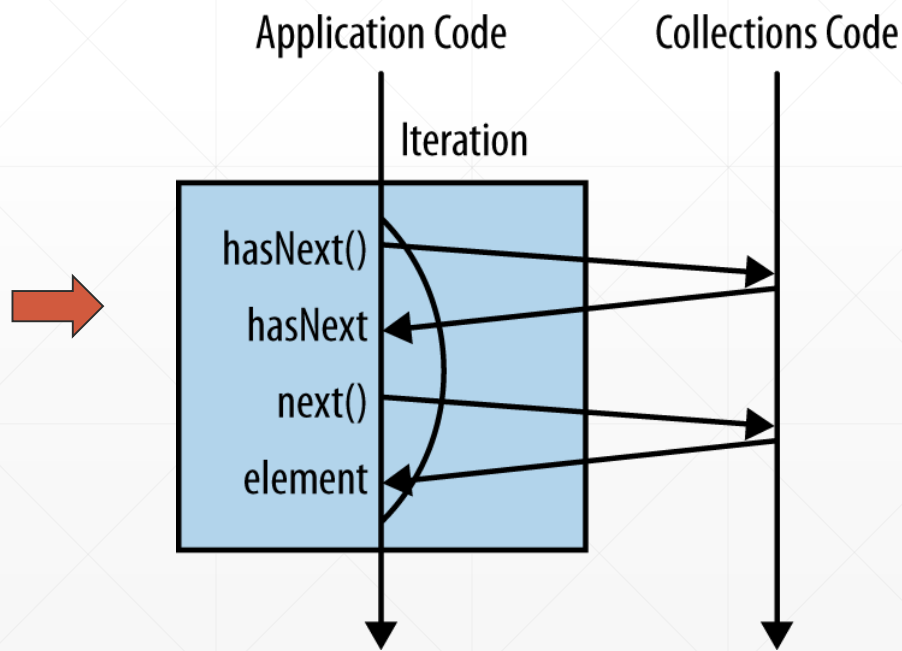
```
// 传统的使用for循环的编程方式
static void filterWithForLoop() {
    1 int count = 0;
    //生成学生对象集合
    List<Student> students = Student.getStudents();
    //遍历集合，统计学生人数
    2 for (Student stu : students) {
        if (stu.getFrom().equals("北京"))
            count++;
    }
    3 //输出结果
    System.out.println("来自于北京有学生有: " + count + "名");
}
```

示例：WhatIsStream.java

传统编程模式存在的问题

- 如果搜索条件改变，必须重写代码
- 如果希望并行处理，则必须重写每个循环
- 要想了解循环干了什么，必须仔细地阅读所有代码

Java的ForEach循环，本质上是通过iterator实现的，所有处理工作，在同一线程、同一循环体内部完成。



如果使用Stream API.....

一句就完成了所有工作：遍历→过滤→统计

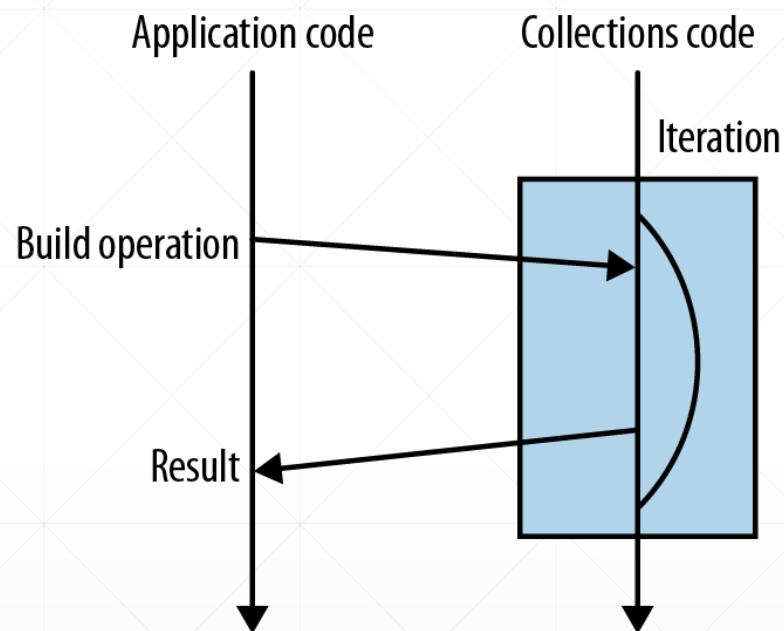
```
// 使用Stream API的编程方式
static void filterWithStreamAPI() {
    //集合对象有一个stream()方法，用于生成流
    long count = Student.getStudents().stream()
        //filter是一个Stream API函数，它可以接收一个Lambda表达式
        .filter(stu -> stu.getFrom().equals("北京"))
        //count是另一个Stream API函数，它接收前一个函数的结果并进行计数
        .count();

    //输出结果
    System.out.println("来自于北京有学生有: " + count + "名");
}
```

上述代码中，stream()方法返回一个“流”，filter和count都是对这个流所施加的“操作”，这种操作可以级联，最后一个才是负责真正启动执行这一连串处理工作的操作（称为“**Terminal Operation**”，本例中是count），中间的称为“**Intermediate Operation**”，它们是被动的，不会引发操作的执行。

Stream API将集合操作“外置”

使用Stream API，调用者只管设定好级联的操作并启动之，所有操作都是由“Stream”完成的，调用者最后直接拿结果就行了。



这种编程方式，只需告诉计算机“Do what”，而无需告诉计算机“How to Do”，具体如何完成由Stream API框架自己决定。

并行处理

```
// 使用Stream API的并行编程方式
static void filterWithParallelStreamAPI() {
    System.out.println("线程: " + Thread.currentThread().getId()
        + "启动处理工作");
    long count = Student
        .getStudents()
        //生成支持并行处理的流
        .parallelStream()
        //取出每个元素, 打印出负责处理它的线程Id
        .peek(stu -> {
            System.out.println("线程" + Thread.currentThread().getId()
                + "正在处理" + stu);
        }).filter(stu -> stu.getFrom().equals("北京")).count();

    System.out.println("线程" + Thread.currentThread().getId() +
        "显示处理结果: 来自于北京有学生有" + count + "名");
}
```

有关Java多线程开发方面的内容, 安装有专门课程介绍。

线程1启动处理工作
线程19正在处理(赵六, 重庆)
线程15正在处理(王五, 北京)
线程16正在处理(朱十, 上海)
线程20正在处理(张三, 北京)
线程17正在处理(李四, 上海)
线程1正在处理(张八, 上海)
线程21正在处理(韩七, 南京)
线程18正在处理(赵九, 北京)
线程1显示处理结果: 来自于北京有学生有3名



不需要与Thread直接打交道!

并行循环

当采用并行方式执行遍历时，元素的访问顺序将变得“混乱”

```
public class ParallelStreams {  
    public static void main(String[] args){  
        System.out.println("创建字符串集合");  
        List<String> strings = new ArrayList<>();  
        for (int i = 0; i < 10; i++) {  
            strings.add("Item " + i);  
        }  
        //乱序显示集合中的元素  
        System.out.println("集合中包容以下元素: ");  
        strings.parallelStream()  
            .forEach(System.out::println);  
    }  
}
```



创建字符串集合
集合中包容以下元素:
Item 6
Item 5
Item 8
Item 1
Item 2
Item 0
Item 9
Item 3
Item 4
Item 7

参看示例 `ParallelStreams.java`

“Stream”的定义

“流（Stream）”：从支持数据处理操作的源生成的元素序列。

元素序列

- 流包容有一组数据，其数目不限

源

- 流从“源（通常是集合）”中“抽取”数据。

操作

- 代表对数据具体的处理工作

Stream API的长处



声明式编程风格，代码简短易读。



JDK中提供的各种流数据处理方法可动态组合，使用灵活方便。



内存占用减少，性能优化。因为无须同时将所有的数据调入内存，并且多个操作可以“打包”到一起运行，从而只遍历一次数据，并且支持并行执行。